SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3

5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3

11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3

18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Code of Conduct:

I Yuvaraj D_192421230 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Yuvaraj.D

1. Testing as an Engineering Activity in a Banking Application

Introduction

Software testing is not simply executing a few test cases to find bugs. It is an **engineering activity** that follows systematic methods, standards, tools, planning and measurement. A banking application requires highly reliable testing because failures can cause financial loss and loss of customer trust.

Problems with Ad-Hoc Testing

The banking team currently follows unstructured testing. This may result in:

- Random execution of test cases
- Important scenarios being missed
- Duplicate testing
- Poor documentation
- Difficulty reproducing defects
- Late detection of critical defects
- Inconsistent testing between releases

Therefore, testing should be redesigned as a systematic engineering process.

Systematic Test Design

The team should first identify requirements and convert them into testable conditions.

For example, for a fund-transfer feature:

Requirement	Test Condition
Transfer ₹10,000	Valid transfer
Transfer above daily limit	Limit validation
Insufficient balance	Error handling
Wrong account number	Account validation
Network failure	Recovery testing
Duplicate transaction	Duplicate prevention

Test cases can then be designed using:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing
- Use-Case Testing

Use of Testing Tools

Automation and management tools can improve consistency.

Examples:

- **Selenium** – web automation
- **Postman** – API testing
- **JMeter** – performance testing
- **Jira** – defect tracking
- **JUnit/TestNG** – unit testing

Automated regression tests can be executed whenever new banking functionality is added.

Testing Standards

The team should establish standards for:

- Test-case format
- Defect severity and priority
- Test execution procedure
- Entry and exit criteria
- Regression testing
- Test documentation
- Review and approval

Practical Example

Suppose a customer transfers ₹50,000.

A systematic test approach checks:

1. Whether the account has sufficient balance.
2. Whether ₹50,000 is within the daily transaction limit.
3. Whether the beneficiary account is valid.
4. Whether authentication/OTP succeeds.
5. Whether the transaction is completed only once.
6. Whether the balance is updated correctly.
7. Whether transaction history is updated.

Advantages

This approach:

- Improves defect detection
- Reduces missed test cases
- Provides repeatable testing
- Improves traceability
- Reduces regression defects
- Increases customer confidence

Conclusion

Testing should be treated as an engineering discipline rather than random bug searching. **Systematic test design, automation, standards, defect tracking and measurable processes** can significantly improve the reliability and quality of banking applications.

2. Structured Testing Process for ATM Withdrawal

Introduction

An ATM requires a well-defined testing process because incorrect withdrawal processing can directly cause financial loss. Testing must be performed throughout the **Software Development Life Cycle (SDLC)**.

Testing Process Across SDLC

1. Requirement Analysis

Testers study requirements such as:

- Valid card
- Correct PIN
- Available balance
- Withdrawal limits
- Cash availability
- Receipt generation

Testers identify ambiguous or missing requirements early.

2. Test Planning

A test plan defines:

- Scope
- Resources
- Testing techniques
- Schedule
- Test environment
- Entry and exit criteria

3. Test Case Design

Test cases are created for normal and abnormal conditions.

Example:

Test Case	Expected Result
Valid card + valid PIN	Login successful
Invalid PIN	Error message
Sufficient balance	Cash dispensed
Insufficient balance	Transaction rejected
Amount above limit	Transaction rejected
ATM has insufficient cash	Appropriate message

4. Test Environment Setup

A controlled ATM simulation environment is prepared with:

- ATM software
- Bank server
- Database
- Card simulator
- Network environment

5. Test Execution

Test cases are executed and actual results are compared with expected results.

6. Defect Reporting

Failures are recorded with:

- Defect ID
- Description
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Screenshots/logs

7. Retesting and Regression Testing

After developers fix a defect, testers retest it. Regression testing ensures that the fix has not affected existing functionality.

Application to ATM Withdrawal

Suppose a customer wants to withdraw ₹10,000.

The process is:

Insert Card → Enter PIN → Select Withdrawal → Enter ₹10,000 → Validate Balance → Validate Limit → Dispense Cash → Update Balance → Print Receipt

Each step is independently tested.

Benefits of Process Consistency

A defined process:

- Reduces missed scenarios
- Improves defect tracking
- Ensures repeatability
- Improves communication
- Reduces regression defects

- Provides better quality control

Expected Outcomes

The ATM system should:

- Correctly authenticate customers
- Prevent unauthorized withdrawals
- Dispense correct cash
- Maintain accurate balances
- Handle failures safely
- Provide correct transaction records

Conclusion

A structured testing process throughout the SDLC ensures that ATM withdrawal functionality is tested systematically. It reduces defects and improves **security, reliability and consistency**.

---

3. Errors, Defects and Failures in a Web Application

Introduction

Errors, defects and failures are related but different concepts in software testing. Understanding these terms helps testers report and resolve problems correctly.

1. Error

An **error** is a human mistake made during requirements, design or coding.

Example:

A developer misunderstands the requirement and assumes that passwords can contain only eight characters.

2. Defect

A defect is an incorrect implementation present in the software because of an error.

Example:

The login program rejects passwords containing more than eight characters even though the requirement allows 20 characters.

3. Failure

A failure occurs when the software actually executes and produces an incorrect result.

Example:

A customer enters a valid 15-character password, but the application displays **"Invalid Password."**

Relationship

Human Error → Defect in Software → Execution → Failure

Login Example

Suppose the correct password is:

```
Deepak@123456
```

The requirement allows passwords up to 20 characters.

If the developer incorrectly writes:

```
password.length <= 8
```

then:

- Developer misunderstanding = **Error**
- Incorrect password validation logic = **Defect**
- User being rejected during login = **Failure**

Transaction Example

Expected transaction amount:

₹5,000

Actual deducted amount:

₹50,000

Possible sequence:

Coding mistake → Incorrect calculation logic → Defect → Wrong amount deducted → Failure

Problems Due to Misunderstanding

If testers confuse these terms:

- Defect reports become unclear
- Developers cannot reproduce problems
- Root causes are missed
- Severity may be incorrectly assigned
- Repeated defects may occur

A good defect report should contain:

1. Defect ID
2. Module
3. Description
4. Preconditions
5. Steps to reproduce
6. Expected result
7. Actual result
8. Severity
9. Priority
10. Environment
11. Screenshots/logs

Conclusion

Understanding **error, defect and failure** provides clarity in software testing. Correct terminology improves communication between developers and testers and makes defect resolution faster and more effective.

---

4. Defect Clustering and Test Prioritization

Introduction

Defect clustering is one of the important principles of software testing. It states that a small number of modules often contain a large percentage of defects.

This is similar to the **Pareto principle**, where a few modules may account for most defects.

Application

Suppose an application contains five modules:

Module	Defects
Login	5
Payment	32
Product	7
Search	4
Profile	2

The payment module contains the majority of defects.

Therefore, testing resources should be concentrated on the payment module.

Test Prioritization

Test cases can be prioritized according to:

- Business importance
- Risk
- Defect history
- Complexity
- Frequency of use
- Security impact
- Customer impact

Priority Levels

High Priority:

- Payment
- Login
- Fund transfer
- Order placement

Medium Priority:

- Search
- Profile update

Low Priority:

- Cosmetic changes

Practical Example

In an e-commerce system, previous releases show repeated payment failures.

Therefore:

1. Execute payment tests first.
2. Perform boundary testing.
3. Test different payment methods.
4. Test network interruption.
5. Test duplicate payments.
6. Perform regression testing.

Other Testing Principles

Exhaustive Testing is Impossible

It is impossible to test every possible input combination.

Testing Shows Presence of Defects

Testing can demonstrate that defects exist but cannot prove that software is completely defect-free.

Repeatedly executing the same tests may stop finding new defects. Test cases must therefore be updated.

Benefits

Defect clustering helps:

- Allocate resources efficiently
- Identify risky modules
- Reduce testing time
- Detect critical defects earlier
- Improve release quality

Conclusion

Defect clustering helps testers focus on high-risk areas rather than distributing testing effort equally across all modules. Combining defect history with **risk-based test prioritization** improves testing efficiency and software quality.

---

5. Defect Classification and Defect Repository

Introduction

A defect repository is a centralized location where software defects are recorded, tracked and analyzed. Simply maintaining a repository is not enough; historical defect information must be used to improve future testing.

Sources of Defects

Defects can originate during:

- Requirement analysis
- Design
- Coding
- Database development
- Integration
- Configuration
- Deployment

Defect Classification

Defects can be classified as:

1. Functional Defects

Feature does not work according to requirements.

2. Performance Defects

System response is slower than expected.

3. Security Defects

Unauthorized access is possible.

4. Interface Defects

Problems occur between modules or systems.

5. Usability Defects

Application is difficult to use.

6. Data Defects

Incorrect or inconsistent data is produced.

Repository Information

Each defect should contain:

- ID
- Module
- Description
- Severity
- Priority
- Root cause
- Status
- Environment
- Developer
- Tester
- Resolution
- Date

Using Repository Data

Suppose the repository shows that payment-related defects occurred repeatedly.

Testers can create additional tests for:

- Payment timeout
- Duplicate transactions
- Incorrect amount
- Network failure
- Refund processing

Root Cause Analysis

Repeated defects should be analyzed using:

- Why-Why analysis
- Fishbone diagram
- Pareto analysis
- Defect trend analysis

Preventing Recurring Defects

The organization should:

- Add regression test cases
- Automate repeated tests
- Conduct code reviews
- Improve requirements
- Perform root-cause analysis
- Update development standards

Expected Quality Improvements

Effective repository usage results in:

- Fewer repeated defects
- Better test coverage
- Faster defect resolution
- Improved process quality
- Reduced maintenance cost

Conclusion

A defect repository should be used as a **learning system**, not merely as a storage system. Historical defect information helps testers predict risky areas and design better tests for future releases.

---

6. Mobile Banking Application Crashes During Fund Transfer

Introduction

Frequent crashes during fund transfers can be caused by coding defects, invalid inputs, database problems, network failures or insufficient error handling. Systematic testing is required to identify the root cause.

Step 1 – Reproduce the Failure

Testers first determine:

- When the crash occurs
- Which device is affected
- Which OS version is affected
- Whether the amount matters
- Whether network conditions matter

Step 2 – Analyze Logs

Application logs, crash reports and server logs are examined.

For example:

Fund Transfer → API Request → Database Update → Application Crash

This indicates that the defect may be related to API/database processing.

Step 3 – Apply Testing Techniques

Boundary Value Testing

Test:

- ₹1
- ₹999
- ₹10,000
- Maximum transfer amount
- ₹0
- Negative amount

Equivalence Partitioning

Inputs can be divided into:

- Valid amount
- Zero
- Negative amount
- Excessive amount
- Non-numeric input

Negative Testing

Test:

- Network disconnection
- Server unavailable
- Invalid beneficiary
- Session timeout
- Duplicate transaction

Practical Example

Suppose the application crashes when transferring ₹50,000 during weak network connectivity.

The tester repeats the test under:

- Strong network
- Weak network
- No network
- Network switching

- Server timeout

If the crash occurs only during timeout, improper exception handling may be the root cause.

Improvements

The team should introduce:

- Automated API tests
- Exception handling
- Crash monitoring
- Regression testing
- Device compatibility testing
- Performance testing
- Security testing

Conclusion

Systematic testing identifies not only the visible crash but also its underlying cause. Combining **functional, boundary, negative, API and regression testing** can prevent repeated failures in mobile banking applications.

---

7. Testing vs Debugging

Introduction

Testing and debugging are related but different activities. Confusing them can cause inefficient defect resolution.

Testing

Testing is performed to:

- Find defects
- Verify requirements
- Validate functionality
- Identify failures

The tester normally does not fix the defect.

Debugging

Debugging is performed mainly by developers to:

- Find the root cause
- Analyze source code
- Correct the defect
- Verify the correction

Difference

Testing	Debugging
Finds failures	Finds causes
Mainly performed by testers	Mainly performed by developers
Compares actual and expected results	Examines code and execution
Produces defect reports	Produces fixes
Quality evaluation	Problem resolution

Example

Expected:

Account balance = ₹20,000

After withdrawal:

Expected = ₹15,000

Actual:

₹14,000

Testing identifies the failure and reports it.

The developer then examines the code and discovers that the withdrawal amount was deducted twice.

That investigation and correction is **debugging**.

Structured Integration

The process should be:

Testing → Defect Report → Debugging → Fix → Retesting → Regression Testing

Problems with Confusion

If testers attempt debugging instead of reporting:

- Defect information may be incomplete
- Root cause may be incorrectly assumed
- Testing time increases

If developers only debug without proper testing:

- Other failures may remain undiscovered

Conclusion

Testing and debugging complement each other. Testing detects failures while debugging identifies and removes their causes. A structured workflow ensures efficient defect resolution and better software quality.

---

8. Cross-Browser Testing for an E-Commerce Platform

Introduction

Web applications may behave differently across browsers because browsers use different rendering engines, JavaScript implementations and CSS support.

Therefore, cross-browser testing is necessary.

Possible Causes of Defects

- CSS incompatibility
- JavaScript differences
- Unsupported browser features
- Responsive design problems
- Different browser versions
- Different screen resolutions

Testing Strategy

The tester should identify important combinations.

Example:

Browser	Desktop	Mobile
Chrome	✓	✓
Edge	✓	✓
Firefox	✓	✓
Safari	✓	✓

Functional Testing

Important functions include:

- Login
- Product search
- Add to cart
- Checkout
- Payment
- Order tracking

Compatibility Testing

Check:

- Layout
- Buttons
- Fonts
- Images
- JavaScript
- Forms
- Pop-ups
- Payment gateway

Automation Tools

Suitable tools include:

- Selenium
- Playwright
- Cypress
- BrowserStack
- LambdaTest

Automation allows the same test suite to run across multiple browsers.

Example

Suppose the **Add to Cart** button works correctly in Chrome but does not respond in Firefox.

Testing identifies browser-specific behavior.

The developer can then investigate JavaScript compatibility.

Benefits

Cross-browser testing improves:

- Compatibility
- User experience
- Reliability
- Customer satisfaction
- Market coverage

Conclusion

A systematic cross-browser strategy combining **compatibility matrices, functional testing and automation** helps ensure that an e-commerce application behaves consistently across browsers and devices.

9. Logical Defects in Payroll Salary Calculation

Introduction

A payroll system must calculate salaries accurately. Logical defects in salary calculations can result in employees being overpaid or underpaid.

Defect Classification

The defects may be classified as:

- Functional defects
- Calculation defects
- Business-rule defects
- Data defects
- Boundary defects

Example

Suppose:

Basic Salary = ₹30,000

HRA = 20%

Expected HRA:

₹30,000 × 20/100 = ₹6,000

If the program calculates ₹3,000, there is a logical/calculation defect.

Possible Causes

- Incorrect formula
- Wrong operator
- Incorrect percentage
- Wrong salary slab
- Incorrect tax rule
- Integer/decimal conversion
- Incorrect database values

Testing Techniques

Boundary Testing

Test salary values around:

- Minimum salary
- Tax threshold

- Maximum salary

Equivalence Partitioning

Salary can be divided into:

- Low salary
- Medium salary
- High salary

Decision Table

Different combinations of:

- Salary
- Tax
- Allowances
- Deductions
- Bonus

can be tested.

Prevention

Future defects can be reduced through:

- Requirement reviews
- Code reviews
- Unit testing
- Automated calculation tests
- Regression testing
- Boundary testing

Example Test

If:

Basic = ₹30,000
HRA = 20%
DA = 10%

Then:

HRA = ₹6,000

DA = ₹3,000

Gross salary:

₹30,000 + ₹6,000 + ₹3,000 = ₹39,000

The test case should verify the calculation automatically.

Conclusion

Classification of payroll defects helps testers understand their causes and select suitable testing techniques. **Boundary testing, decision tables, unit testing and automated regression testing** can greatly reduce salary calculation errors.

---

10. Structured Test Case Design Instead of Random Testing

Introduction

Random testing without planning can miss important scenarios. Testing should be treated as an engineering activity where test cases are designed systematically from requirements and risks.

Problems with Random Testing

- Important cases may be missed
- No traceability
- Duplicate tests
- Difficult to measure coverage
- Poor defect reproduction
- Inconsistent results

Structured Test Case Design

Each test case should contain:

1. Test Case ID
2. Requirement ID
3. Description
4. Preconditions
5. Test data
6. Steps
7. Expected result
8. Actual result
9. Status

Example – Login

TC001: Valid login

Input:

Username = Deepak
Password = Correct password

Expected:

User successfully enters dashboard.

TC002: Invalid password

Expected:

Error message displayed.

Testing Techniques

Equivalence Partitioning

Divide input into valid and invalid groups.

Boundary Value Analysis

Test values at and around limits.

Decision Tables

Useful when several conditions affect the output.

State Transition Testing

Useful for login states such as:

Logged Out → Login → Logged In → Logout

Use-Case Testing

Tests complete user workflows.

Benefits

Structured test design provides:

- Better coverage
- Higher defect detection
- Traceability
- Repeatability
- Easier maintenance
- Better regression testing

Conclusion

Replacing random testing with structured test-case design improves the effectiveness of testing. Using **equivalence partitioning, boundary analysis, decision tables and state transitions** ensures systematic defect detection.

11. Data Inconsistency in Healthcare Application

Introduction

Healthcare applications handle sensitive patient information. Data inconsistency between modules can result in incorrect diagnosis, billing or treatment information.

Possible Sources

Defects may originate from:

- Incorrect database design
- Coding errors
- Incorrect API mapping
- Synchronization problems
- Transaction failures
- Incorrect validation
- Concurrent updates

Example

Patient's age in the registration module:

25 years

But in the billing module:

52 years

This indicates a data consistency problem.

Testing Approach

Database Testing

Verify:

- Data accuracy
- Constraints
- Relationships
- Primary keys
- Foreign keys

Integration Testing

Check data transfer between:

Registration → Medical Records → Billing

Verify that APIs transfer correct data.

Transaction Testing

Ensure that database updates are atomic.

Test Case

1. Register patient.
2. Update patient information.
3. Open medical records.
4. Open billing.
5. Compare patient information across modules.

Expected result:

All modules should display consistent information.

Defect Tracking

Reports should contain:

- Patient/module information
- Steps
- Expected value
- Actual value
- Database evidence
- API response
- Severity

Conclusion

Systematic **database, API, integration and transaction testing** helps maintain healthcare data integrity. Clear defect tracking makes data-related problems easier to identify and resolve.

---

12. Repeated Defects Using a Defect Repository

Introduction

Repeated defects indicate that previous defect information is not being effectively used. A defect repository can provide historical information that improves future testing.

Repository Analysis

Testers should analyze:

- Defects by module
- Defects by severity
- Defects by release
- Defects by root cause
- Reopened defects
- Repeated defects

Example

Suppose three releases show:

Release	Payment Defects
R1	12
R2	15
R3	18

This indicates that payment is a high-risk area.

Pattern Identification

Patterns can be identified using:

- Pareto analysis
- Defect trend charts
- Root-cause analysis
- Module-wise analysis
- Severity analysis

Improving Test Design

If payment defects repeatedly involve network interruptions, new test cases should cover:

- Network loss
- Timeout
- Retry
- Duplicate payment
- Server failure
- Payment recovery

Prevention

- Add regression tests
- Automate high-risk tests
- Conduct root-cause analysis
- Improve coding standards
- Review requirements
- Perform code reviews

Conclusion

A defect repository should be actively analyzed to identify patterns. Historical defects can be converted into **new regression tests and preventive actions**, reducing repeated defects across releases.

---

13. Web Learning System Fails During Peak Usage

Introduction

A web-based learning system may work correctly under normal load but fail when thousands of students access it simultaneously. This indicates insufficient performance testing.

Testing Gaps

Possible missing tests include:

- Load testing
- Stress testing
- Spike testing
- Endurance testing
- Capacity testing

Importance of Test Planning

The test plan should define:

- Expected number of users
- Peak load
- Response time
- Server resources
- Database capacity
- Performance requirements

Example

Normal:

500 users → 2 seconds response

Peak:

5,000 users → system crashes

This shows that the system was not tested for realistic peak usage.

Performance Testing

Tools such as:

- JMeter
- LoadRunner
- Gatling

can simulate large numbers of users.

Test Scenarios

Test:

1. 500 users
2. 1,000 users
3. 2,500 users
4. 5,000 users
5. Sudden spike to 10,000 users

Measure:

- Response time
- Throughput
- Error rate
- CPU usage
- Memory usage

Improvements

- Capacity planning
- Database optimization
- Caching
- Load balancing
- Performance monitoring
- Stress testing before release

Conclusion

Proper test planning and performance coverage are essential for systems expected to handle large numbers of users. Load, stress and spike testing can identify failures before deployment.

---

14. Early Involvement of Testers in SDLC

Introduction

Testers should not be involved only after coding is completed. Early tester involvement follows the principle of **early testing**, where defects are identified as soon as possible.

Role During SDLC

Requirement Phase

Testers:

- Review requirements
- Identify ambiguity
- Create acceptance criteria
- Identify missing conditions

Design Phase

Testers review:

- Architecture
- Interfaces
- Data flow
- Security requirements

Development Phase

Testers prepare:

- Test cases
- Test data
- Automation scripts

Testing Phase

They execute:

- Functional tests
- Integration tests
- System tests
- Regression tests

Practical Example

Suppose a banking requirement says:

"A customer can transfer money up to the daily limit."

The tester asks:

- What is the exact limit?
- What happens at the exact limit?
- What happens above the limit?
- What happens after multiple transfers?

These questions can prevent requirement defects before coding begins.

Benefits

Early tester involvement:

- Reduces development cost
- Prevents requirement defects
- Improves testability
- Improves communication
- Reduces late-stage defects

Collaboration

Teams should conduct:

- Requirement reviews
- Design reviews
- Daily discussions
- Joint defect analysis
- Sprint testing

Conclusion

Testers should be involved throughout the SDLC rather than only during final testing. Early involvement enables **early defect prevention, better requirements and improved software quality**.

---

15. Incomplete Test Coverage

Introduction

A system may work correctly for tested scenarios but fail in untested conditions. This happens when test coverage is incomplete.

Example

An online payment system is tested for:

₹100, ₹500 and ₹1,000

But no test is performed for:

- ₹0
- Negative amount
- Maximum amount
- Decimal amount
- Network interruption

The system may fail in these cases.

Identifying Coverage Gaps

Testers should compare:

Requirements → Test Cases → Execution Results

This provides traceability.

Improving Test Design

Use:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Tables
- State Transition Testing
- Use-Case Testing
- Risk-based testing

Example

For an age field accepting 18–60:

Test:

17, 18, 19, 59, 60, 61

This is better than testing only 25 and 30.

Coverage Types

- Requirement coverage
- Functional coverage
- Code coverage
- Risk coverage
- Scenario coverage

Benefits

Better coverage:

- Finds more defects
- Reduces production failures
- Improves confidence
- Ensures important requirements are tested

Conclusion

Complete test coverage does not mean testing every possible input. Instead, it means systematically covering **important requirements, risks, boundaries and user scenarios**.

16. Improving Defect Reporting in Logistics Application

Introduction

Poor defect reporting can delay defect identification and resolution. A clear and standardized defect report allows developers and testers to understand the problem quickly.

Good Defect Report

A defect report should contain:

1. Defect ID
2. Title
3. Module
4. Environment
5. Preconditions
6. Steps to reproduce
7. Expected result
8. Actual result
9. Severity
10. Priority
11. Attachments
12. Logs

Example

Title: Delivery status not updated after shipment confirmation.

Steps:

1. Login.
2. Open order.
3. Confirm shipment.
4. Refresh order status.

Expected:

Status = Shipped

Actual:

Status = Processing

Severity

Severity indicates the technical/business impact.

Example:

- Critical
- High
- Medium
- Low

Priority

Priority indicates how quickly the defect should be fixed.

A low-severity defect may still have high priority if it affects many customers.

Benefits

Structured reports:

- Reduce communication gaps
- Improve reproduction
- Speed up debugging
- Improve tracking
- Reduce duplicate defects

Conclusion

Clear defect definitions and standardized reporting improve communication between testers and developers. A structured defect report enables faster resolution and better software quality.

---

17. Process-Based Testing for Consistency

Introduction

Testing different modules using different informal approaches can produce inconsistent quality. A process-based testing approach defines common procedures, standards and measurements.

Standard Testing Process

Requirement Analysis → Test Planning → Test Design → Environment Setup → Execution → Defect Reporting → Retesting → Regression → Closure

Standardization

All modules should follow common standards for:

- Test-case format
- Naming conventions
- Defect severity
- Priority
- Test execution
- Documentation

- Reporting

Example

Suppose the Login and Payment modules use different defect classifications.

Login uses:

High / Medium / Low

Payment uses:

P1 / P2 / P3 / P4

This creates confusion.

A common standard should be adopted.

Process-Based Approach

For every module:

1. Identify requirements.
2. Perform risk analysis.
3. Design test cases.
4. Execute tests.
5. Record defects.
6. Retest fixes.
7. Perform regression testing.
8. Measure coverage.

Benefits

- Consistent quality
- Better traceability
- Easier training
- Reduced duplication
- Better management
- Improved defect detection

Conclusion

Standardized process-based testing ensures that every module is tested using consistent methods. It improves **repeatability, quality, documentation and defect management**.

---

18. Untested Edge Cases and Boundary Testing

Introduction

Edge cases occur near the limits of valid input. They are often missed because testers mainly focus on normal conditions.

Example

Suppose an ATM allows withdrawal from:

₹100 to ₹20,000

Important tests include:

- ₹99
- ₹100
- ₹101
- ₹19,999
- ₹20,000
- ₹20,001

Boundary Value Analysis

Boundary Value Analysis focuses on:

Minimum − 1, Minimum, Minimum + 1

and

Maximum − 1, Maximum, Maximum + 1

Why Edge Cases Matter

Many programming errors occur because of:

- Incorrect comparison operators
- Off-by-one errors
- Overflow
- Incorrect validation
- Unexpected null values

Practical Example

Requirement:

Age must be 18–60.

Test:

- 17 → Invalid
- 18 → Valid
- 19 → Valid
- 59 → Valid
- 60 → Valid
- 61 → Invalid

Improvements

Testers should:

- Identify input boundaries
- Include negative scenarios
- Review requirements
- Use boundary-value techniques
- Automate edge-case tests

Conclusion

Edge cases must be deliberately included in test design. Boundary Value Analysis is especially effective for detecting defects near minimum and maximum limits and helps prevent failures after deployment.

---

19. Defect Repository and Documentation

Introduction

Proper defect documentation is essential for tracking, analysis and prevention. A defect repository provides a centralized location for storing information about software defects.

Required Information

Each defect should record:

- Unique ID
- Title
- Module
- Description
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Environment
- Status
- Root cause
- Resolution
- Tester
- Developer

Defect Life Cycle

New → Assigned → Open → Fixed → Retest → Verified → Closed

If the defect still exists:

Retest → Reopened

Repository Benefits

It helps to:

- Track defect status
- Identify repeated defects
- Analyze risky modules
- Measure quality
- Support regression testing
- Maintain historical information

Example

Suppose the repository shows 30 defects in the payment module over three releases.

This indicates that payment is a high-risk area.

Testers can increase:

- Payment regression tests
- API testing
- Security testing
- Negative testing
- Performance testing

Documentation Improvements

Use:

- Standard templates
- Mandatory fields
- Severity guidelines
- Screenshots/logs
- Root-cause fields
- Consistent terminology

Conclusion

A well-maintained defect repository transforms defect information into useful quality data. Proper documentation improves **tracking, analysis, communication and prevention of recurring defects**.

20. Redesigning Testing for a Banking Transaction System

Introduction

Banking transaction systems require extremely high reliability. Repeated transaction failures indicate weaknesses in the existing testing process.

Testing should be redesigned using a systematic engineering approach.

1. Requirement Analysis

Identify requirements for:

- Login
- Account balance
- Fund transfer
- Deposit
- Withdrawal
- Transaction limits
- Authentication
- Transaction history

2. Risk Analysis

High-risk functions should receive more testing.

For example:

Fund Transfer → High Risk

because failure can cause financial loss.

3. Test Case Design

Use:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Tables
- State Transition Testing
- Use-Case Testing

Example – Fund Transfer

Test:

1. Valid transfer

2. Insufficient balance
3. Invalid account
4. Maximum transfer amount
5. Amount above limit
6. Network failure
7. Duplicate request
8. Session timeout
9. Server failure

4. Automation

Automate repetitive regression tests.

Example:

After every release, automatically test:

Login → Balance → Transfer → Transaction History

5. Defect Management

Every defect should include:

- Reproduction steps
- Expected result
- Actual result
- Severity
- Priority
- Logs
- Root cause

6. Regression Testing

After fixing transaction defects, previously working transaction scenarios must be executed again.

7. Performance and Security Testing

Banking systems should also be tested for:

- High transaction loads
- Concurrent users
- Authentication
- Authorization
- Session management
- Data protection

Practical Example

Suppose a customer transfers ₹5,000 but the system deducts ₹5,000 twice because the user presses the transfer button twice during a slow network.

Testing should reproduce:

Slow network + repeated click + same transaction

The system should process the transaction only once.

This can be addressed using:

- Idempotency
- Duplicate transaction detection
- Proper transaction handling
- API testing
- Concurrency testing

Benefits

Structured testing provides:

- Higher defect detection
- Reduced production failures
- Better security
- Improved reliability
- Better customer confidence
- Reduced maintenance cost

Conclusion

Banking transaction testing must be systematic and risk-based. Combining **structured test design, automation, regression testing, performance testing, security testing and defect analysis** can significantly improve transaction reliability and software quality.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand